

Rapport de projet de Conception Agile

Application Planning Poker - M1 Informatique

Grigori GRATCHEV

Gaston POTHIN

1. Introduction et Contexte

Ce projet d'application constitue le projet final du cours "Conception agile de projets informatiques" et a pour objectif de réaliser une application de planning poker. Il s'agit de développer une plateforme permettant à des joueurs de mener une partie de planning poker, tout en respectant les règles et les principes vus en cours.

L'équipe est constituée de Grigori GRATCHEV et Gaston POTHIN. L'organisation du travail s'est étalée sur 7 semaines, du 3 novembre au 18 décembre. Nous avons tenté de travailler le plus possible en pair programming lors des séances de cours mais l'organisation du travail s'est surtout faite en Git Flow où chacun a travaillé sur sa branche.

La planification prévue était la suivante :

- Semaine 1 : création du projet et mise en place du dépôt GitHub.
- Semaine 2 : implémentation du menu (choix du pseudo, du nombre de joueurs et du mode de jeu).
- Semaine 3 : création et gestion des parties (génération et utilisation d'un lien pour rejoindre une partie).
- Semaine 4 : import du fichier backlog au format JSON et affichage des fonctionnalités à estimer.
- Semaine 5 : mise en place de la fonctionnalité de vote et calcul des résultats selon le mode de jeu choisi.
- Semaine 6 : sauvegarde des résultats et gestion du fichier JSON de progression.
- Semaine 7 : corrections, finalisation et rédaction de la documentation.

2. Choix Techniques et Architecture

Architecture Logicielle

Nous voulions que notre projet soit disponible et jouable en ligne via un lien à l'instar de Kahoot ou JKLM.fun, alors nous avons retenu une architecture client-serveur.

Langage et Frameworks

Nous avons choisi de réaliser ce projet avec une interface web construite avec react (vite) et un backend développé en python (flask).

Ce choix nous a permis de bien séparer le backend, la partie du code qui gère le fonctionnement du jeu (création des parties, gestion des tours, application des règles, traitement des données) du frontend, la partie interface utilisateur, qui s'occupe de l'affichage à l'écran et des interactions avec les joueurs.

Le backend en Python nous a semblé adapté pour le besoin, d'une part parce que le langage est relativement simple à prendre en main et lisible. Ce choix est également lié aux expériences précédentes : lors d'un stage, nous avons déjà travaillé sur des applications web avec un backend Python et interface en React.js, ce qui nous a permis de réutiliser des habitudes de développement et de gagner du temps. De plus, cette combinaison que nous avons choisie est particulièrement pratique pour faire évoluer l'interface au besoin et de nombreux guides et tutoriels existent pour cette combinaison.

Gestion des données

En créant une partie l'hôte peut choisir d'importer sa liste de tâches qui sera ensuite utilisée lors du jeu. Après la partie, les joueurs ayant participé au jeu peuvent également télécharger les résultats en format json.

The image contains two screenshots of a web application interface. The left screenshot, titled 'Créer une partie', shows a form for creating a new game. It includes fields for 'Pseudo' (with a placeholder 'Entrez votre pseudo'), 'Mode de vote' (set to 'Unanimité'), and 'Temps de vote' (set to '30 secondes'). There is also a section for 'Importer des tâches (JSON)' with a button labeled 'Choisir un fichier' and a status 'Aucun fichier choisi'. A 'Valider' button is at the bottom. The right screenshot, titled 'Partie 3613', shows the end of a game. It displays 'Mode de jeu : unanimité' and 'Votre pseudo : Grigori'. Below, it says 'Partie terminée !' and 'Toutes les taches ont été estimées.' A button labeled 'Télécharger les résultats (JSON)' is highlighted with a red box and an arrow.

Format JSON attendu

Pour importer des tâches le json doit être formaté de cette façon :

```
[
  {
    "nom": "tache 1",
    "description": "Description 1"
  },
  {
    "nom": "tache 2",
    "description": "Description 2"
  }
]
```

Les json générés par ce projet à la suite d'une pause café sont également importables à la création d'une partie, ce qui permet de reprendre la partie là où elle s'était arrêtée.

3. Mise en place de l'intégration Continue

Génération de documentation

La documentation Doxygen est générée automatiquement le workflow suivant ci-dessous.

Workflow

Le déclenchement de la pipeline se fait à chaque push sur la branche main.

Etapes :

- Récupération du code (checkout)
- Installation de Doxygen
- Installation de Graphviz (pour la création de diagrammes)
- Exécution de Doxygen
- Publication de la documentation sur Github Pages

A l'issue de l'exécution ce workflow la documentation est accessible sur la page

<https://ggratchev.github.io/PlanningPoker/>

Tests Unitaires

Workflow

Le déclenchement se fait à chaque push ou pull request sur la branche main.

Etapes:

- Récupération du code (checkout)
- Set up de python
- Installation des dépendances (pip, pytest et pytest-html)
- Exécution des tests

L'utilisation de ce workflow aurait permis la validation automatique du code à chaque changement mais nous ne l'avons pas utilisé. Nous avons commencé à coder trop vite et avons oublié que les tests doivent être mis en place avant de commencer à coder. Leur implémentation à la fin du projet n'aurait pas fait de sens.

4. Manuel Utilisateur et Fonctionnalités

Installation et lancement

Voici les différentes étapes pour lancer le projet en local :

- Cloner le projet

```
git clone https://github.com/ggratchev/PlanningPoker.git
```

Lancement du frontend

- Se déplacer dans le dossier frontend

```
cd frontend
```

- Installer les dépendances

```
npm install
```

- Démarrer le serveur

```
npm run dev
```

Lancement du backend

Il faut tout d'abord commenter toutes les lignes contenant `planningpoker-0aph.onrender.com` et décommenter celles contenant `localhost:5000` car sinon le backend utilisé sera celui hébergé sur `render.com` et ne sera pas lancé en local.

```
const response = await fetch('http://localhost:5000/api/creer-partie', {  
//const response = await fetch('https://planningpoker-0aph.onrender.com/api/creer-partie',  
{
```

- Puis se rendre dans le dossier backend

```
cd backend
```

- Créer un environnement virtuel Python

```
python -m venv venv
```

- Activer l'environnement virtuel

```
venv\Scripts\activate
```

- Installer les dépendances

```
pip install -r requirements.txt
```

- Lancer le serveur Flask

```
python app.py
```

Le projet sera ensuite accessible sur `http://localhost:5173`

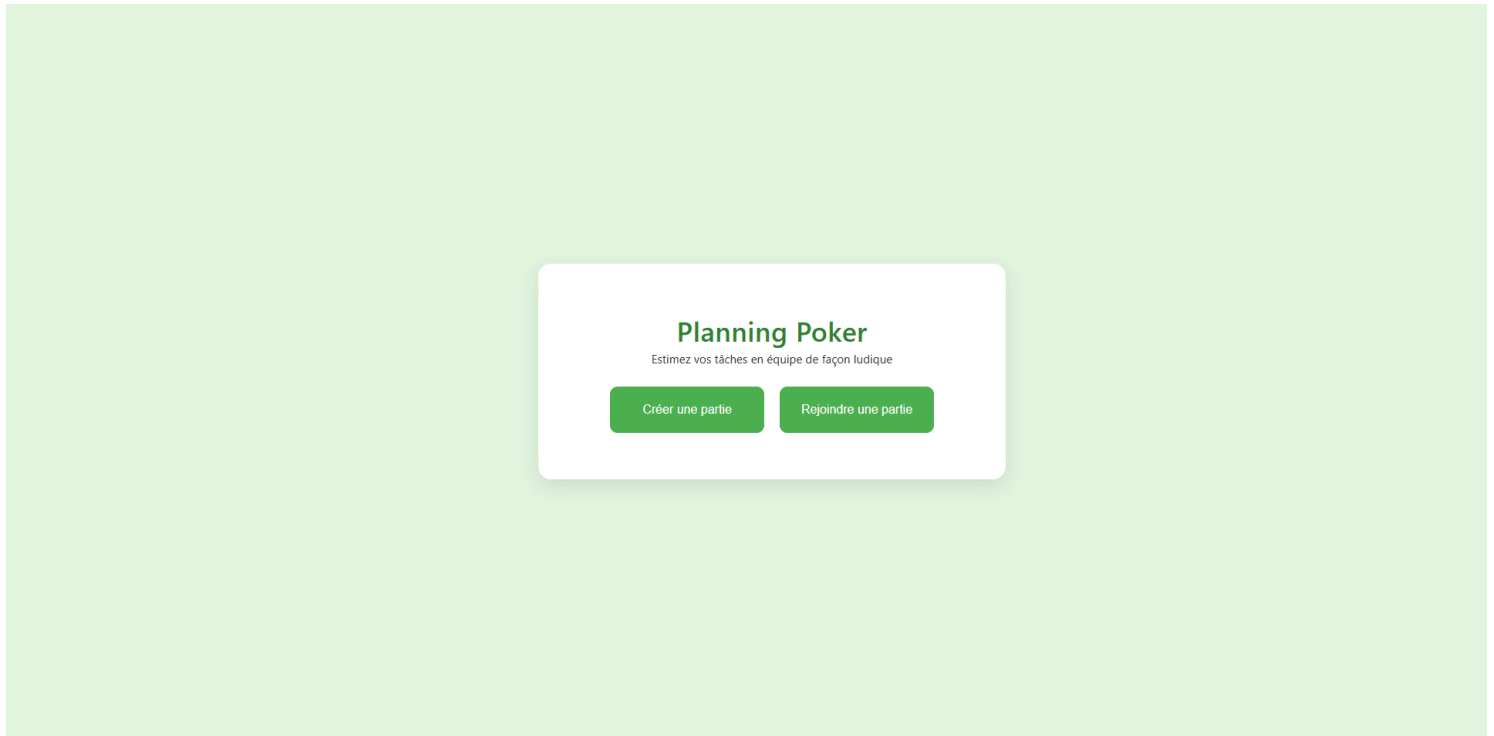
Modes de jeu

Les modes de jeu suivants ont été implémenté :

- **Unanimité** : mode de jeu par défaut. Tant que tous les joueurs n'ont pas voté la même chose pour la tâche actuelle, le vote recommencera.
- **Médiane** : mode de jeu plus rapide. On passera au prochain vote même si des joueurs n'ont pas voté la même chose. La difficulté retenue sera la valeur du milieu si le nombre de joueurs est impair. Si le nombre de joueurs est pair alors la valeur retenue sera la moyenne des 2 valeurs centrales.

Interface

Page d'accueil :



Page de création d'une partie :

Retour à l'accueil

Créer une partie

Pseudo

Grigori

Mode de vote

Unanimité

Temps de vote

30 secondes

Importer des tâches (JSON)

Choose File

taches.json

2 tâche(s) importée(s)

Valider

Page pour rejoindre une partie :

Retour à l'accueil

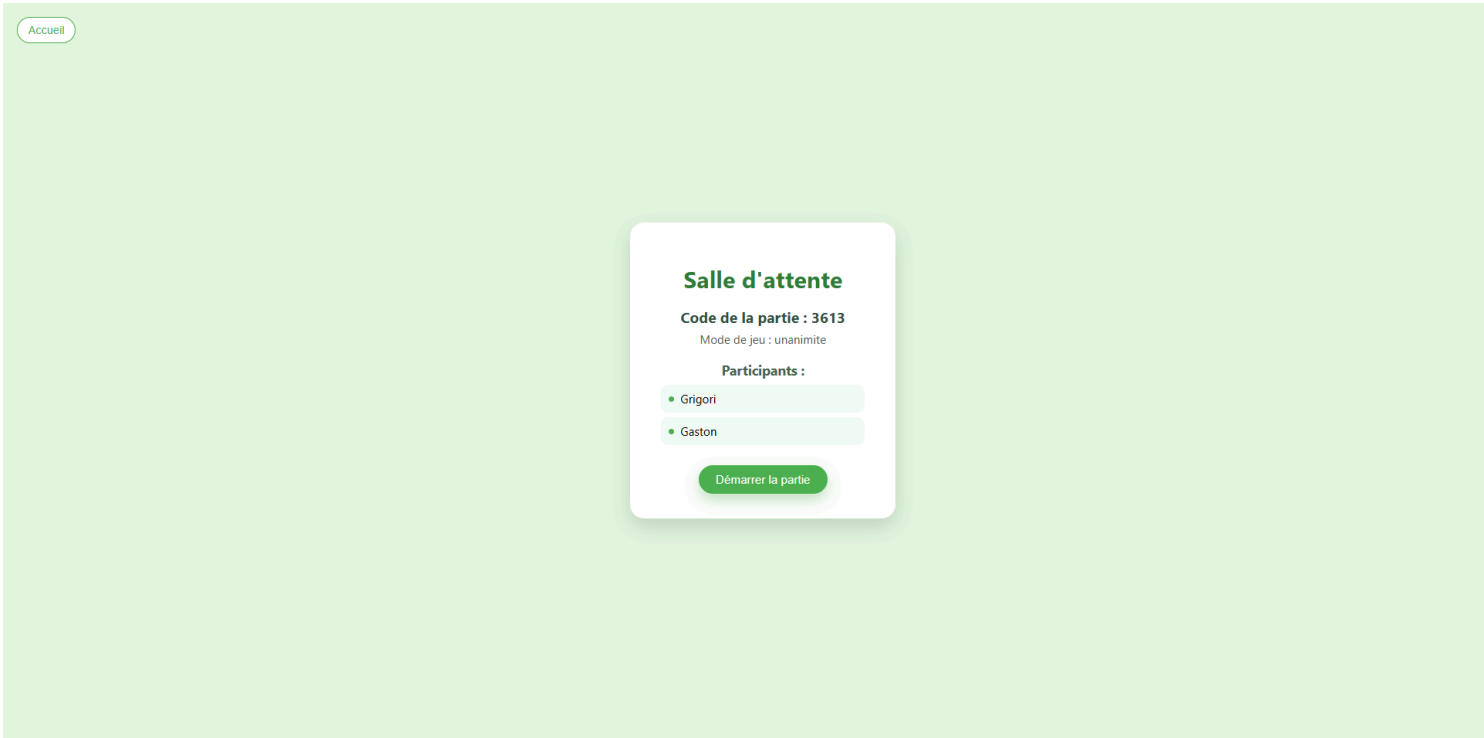
Rejoindre une partie

3613

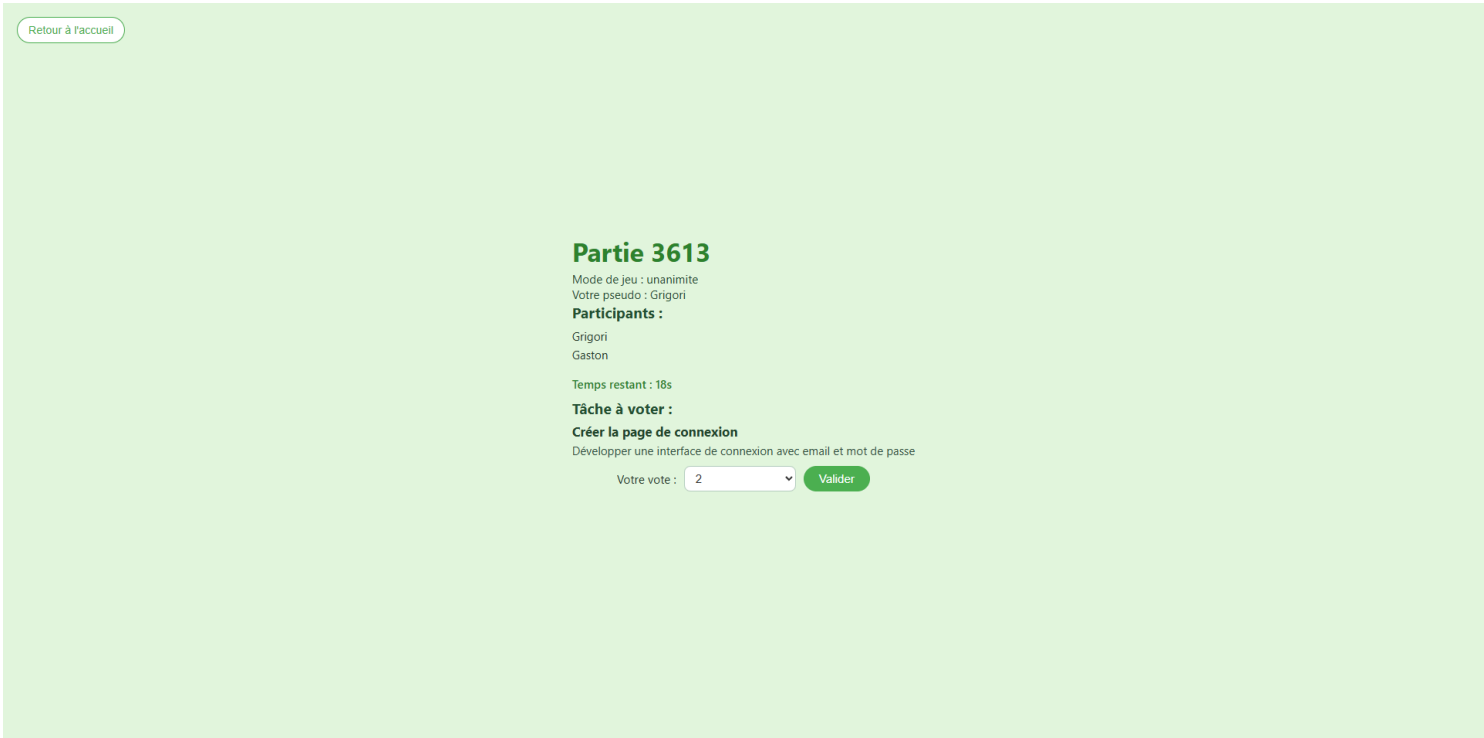
Gaston

Valider

Page Lobby/Salle d'attente avant le début d'une partie :



Page de jeu (lorsqu'une partie est en cours) :



Page de consultation des votes (lorsque tout le monde a voté pour la tâche actuelle):

[Retour à l'accueil](#)

Partie 3613

Mode de jeu : unanimité
Votre pseudo : Grigori

Résultats des votes :

Créer la page de connexion

Développer une interface de connexion avec email et mot de passe

Grigori : 1
Gaston : 1

Gaston : 1

Page de fin de partie :

[Retour à l'accueil](#)

Partie 3613

Mode de jeu : unanimité
Votre pseudo : Grigori

Partie terminée !

Toutes les taches ont été estimées.

[Télécharger les résultats \(JSON\)](#)